eBPF'26 · 개별연구 6주차 논문 리뷰

XDP_CLONE

Efficient In-Driver Packet Duplication for eBPF-Accelerated Systems

Marco Mazzella et al.

26.10.07 박수현

목차

1 배경 및 문제 정의 — 패킷 수신 구조 · 기존 복제 방법

2 제안 기법 — XDP_CLONE 동작 구조

3 평가 — 응용 사례 · 실험 결과 분석

1. 배경 및 문제 정의: 패킷 수신 구조

드라이버 poll() NIC 드라이버 코드

netif_receive_skb() 이후 공통 커널 코드



DROP · 버퍼 재활용

TX · REDIRECT · 전송 큐

→ 다음 디스크립터

XDP 리턴 코드 → sk_buff 생성 여부 결정

PASS 외 경로: 할당·스택 비용 없이 다음 패킷

1. 배경 및 문제 정의: 기존 복제 방법



① bpf_redirect_map() skb 생성 전

동작

BROADCAST 플래그 + DEVMAP → XDP_REDIRECT

맵의 모든 인터페이스로 복제·전송

한계

복사본 내용 전부 동일 · 인터페이스당 1개

원본 로컬 스택 사용 불가

② bpf_clone_redirect() skb 생성 후

동작

대상마다 헤더 수정 → skb 복제 → 전송

원본 로컬 스택 사용 가능

한계

복사본마다 sk_buff 할당 (클론당 ~700ns)

필요한 것: XDP 단계 비용 + 복사본별 수정 + 원본 로컬 사용 → XDP_CLONE

2. 제안 기법: XDP_CLONE

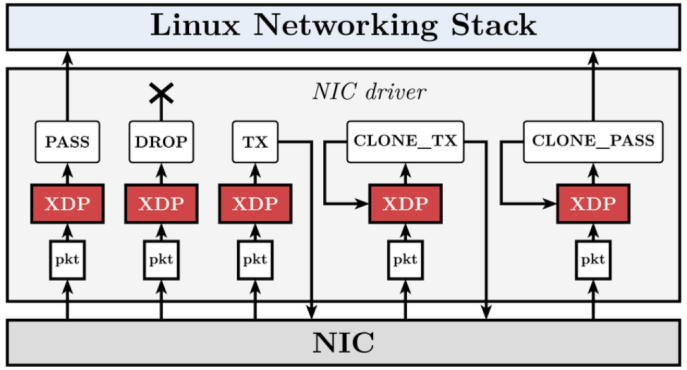
XDP 프로그램
기존 명령어 그대로

return
XDP_CLONE_TX(n)
XDP_CLONE_PASS(n)

드라이버 새 리턴 코드 분기 추가 · 수정한 부분

원본
CLONE_PASS · skb 생성 → 로컬 스택
CLONE_TX · 그대로 전송

복사본 ×n
패킷 복사 → xdp_buff로 감싸 XDP 재실행 → 복사본별 헤더 수정 → 전송



논문 그림 · 기존 리턴 코드와 CLONE 비교

복사본: **같은 XDP 프로그램으로 재진입** → 개별 처리

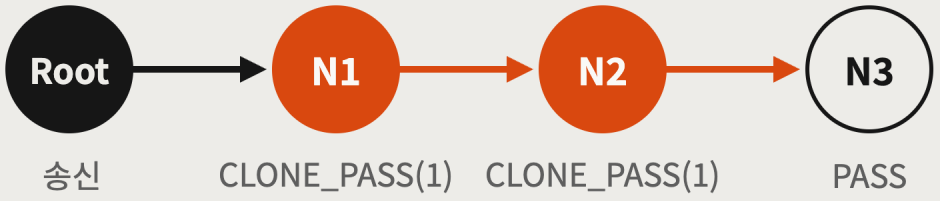
PASS·TX: **원본의 행선지** (로컬 스택 / NIC)

XDP **1-in-1-out 구조 유지**, 반복 실행은 드라이버 담당

3. 평가: XuDP 브로드캐스트

MPI broadcast · 루트 데이터를 참여 노드 전원에게 동일 전송 · 각 노드는 소비 + 전달 → **CLONE_PASS**

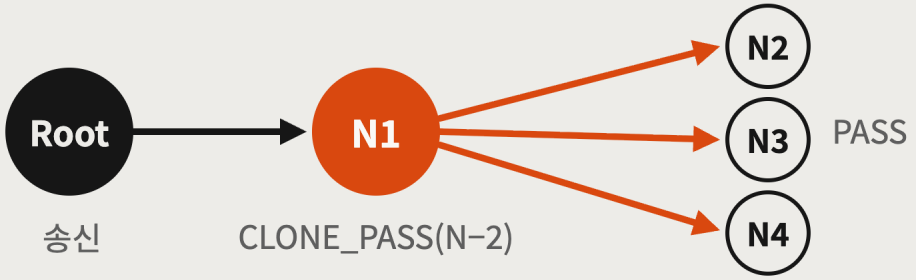
Ring



노드마다 복제 1개 → 부하 균등 분산

다음 노드 주소만 필요 · 홉 N−1번 → 지연 증가

Linear



첫 수신 노드가 나머지 전원에 복제 → 홉 2번

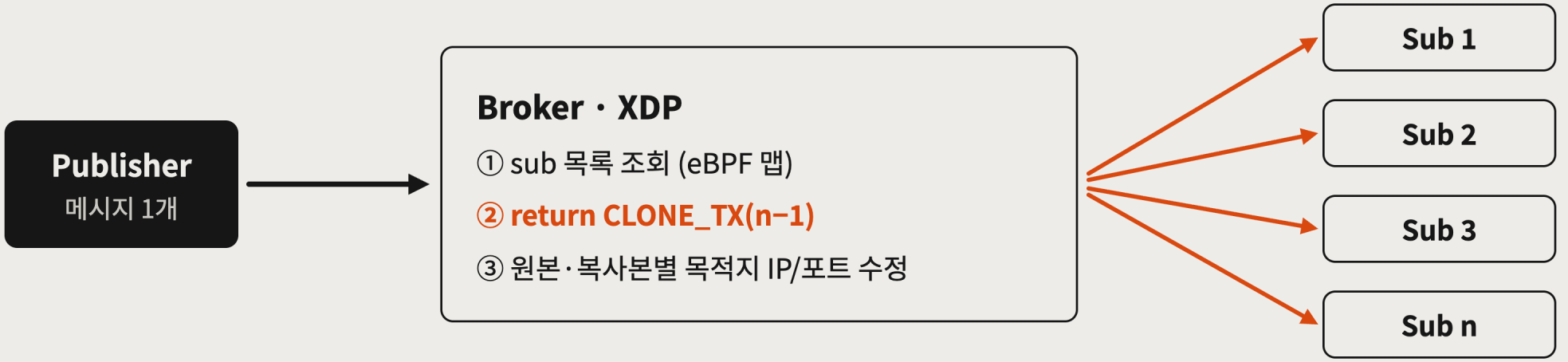
복제 부담 첫 노드 집중 · 전체 명단 필요

복제 시작점: 루트가 아닌 첫 수신 노드

XDP는 수신(RX) 경로에서만 실행 → 루트의 송신 패킷에는 미적용

3. 평가: Pub-Sub 브로커

Pub-Sub · 발행자 메시지 1개를 구독자 전원에게 fan-out



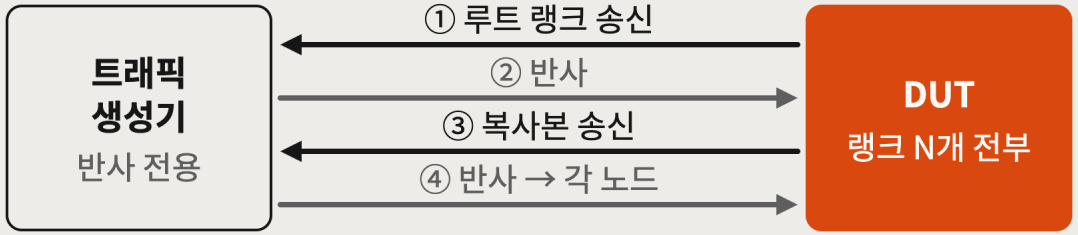
브로커: 메시지 미소비 → CLONE_TX, skb 생성 0개

XuDP 노드: 소비 + 전달 → CLONE_PASS

3. 평가: 실험 환경

DUT · Device Under Test, 성능 측정 대상 머신 1대

XuDP



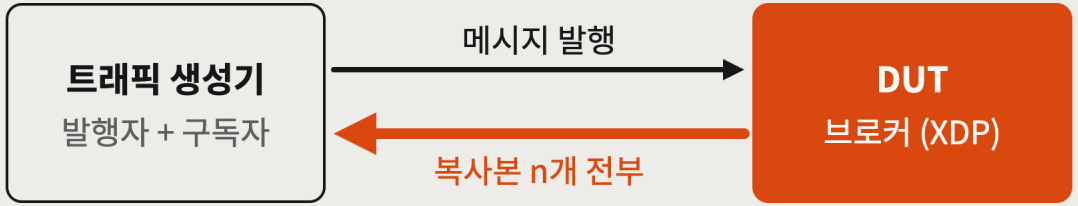
Ring · Linear 두 구조 모두 실험

같은 머신 간 전송 = loopback → NIC·XDP 미경유

생성기는 반사만 → 모든 전송이 DUT RX로 재진입

측정: 노드별 수신 시간 중 가장 늦은 값

Pub-Sub 브로커



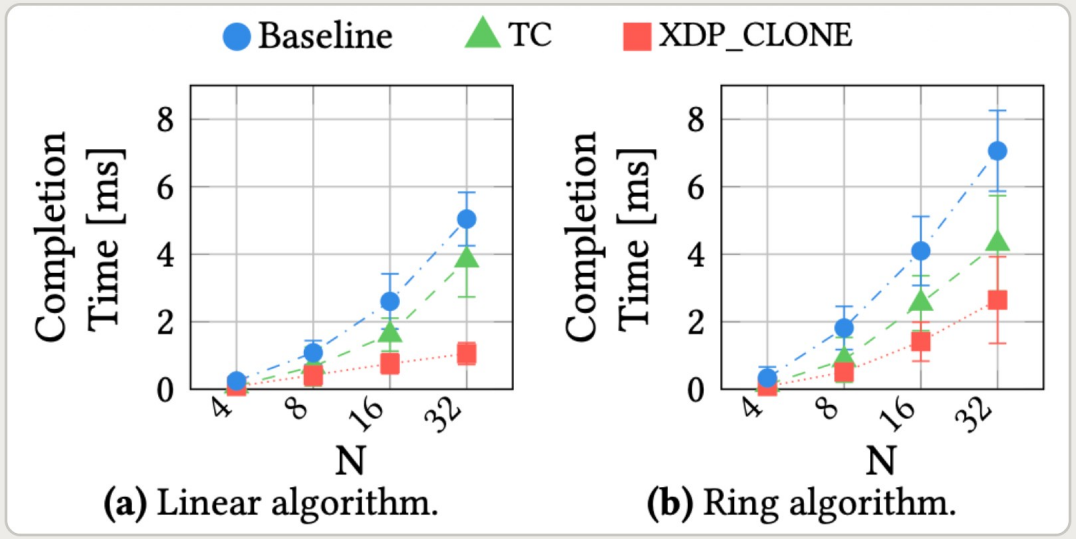
구독자 n명 주소 전부 생성기로 지정

지연: 발행 → 마지막 복사본 도착

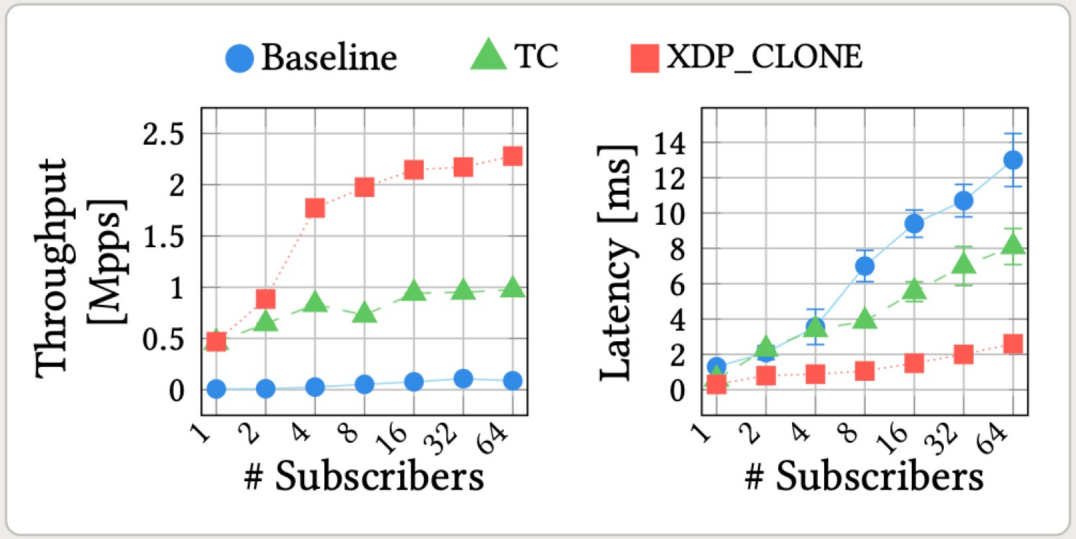
처리량: 복사본 무손실 최대 전송률

3. 평가: 완료 시간 · 처리량

XuDP 브로드캐스트 · 완료 시간



Pub-Sub 브로커 · 처리량 / 지연



복제 수 증가 → TC와 격차 확대: 복사본별 skb 할당 비용 제거

복제가 적을 때(N=4, 구독자 1명)는 세 방식 차이 미미